第 1 页：从梯度下降到共轭梯度

今天我们从一个具体问题出发：一个区域被划分成很多网格以后，怎样高效求出每个位置的未知量？这个问题会带我们走到一个稀疏线性系统，再走到梯度下降、共轭梯度和预条件。

我希望大家最后能把 3 件事联系起来。第 1，矩阵的特征值怎样影响迭代速度；第 2，共轭梯度怎样利用已经完成的搜索；第 3，怎样把问题本身的结构变成更快的求解方法。最后我们还会看实际代码、计算成本和结果检查，让这条路线落到可以运行的求解过程上。

第 2 页：从一张稳态场，得到一个线性系统

先看一个定义在单位正方形上的稳态问题。未知函数是 u，右边的 f 是源项，四周边界上的 u 都等于 0。方程里，横向和纵向的二阶导数系数不同。这里 epsilon 取 0.01，所以纵向的耦合比横向弱。

左图画的是源项，右图画的是网格上的数值解。为了能直接检查离散过程，我们先选择一个已知的正弦函数作为解析解，再把它代回微分方程，计算对应的源项。这样，离散求出来的场就有一个明确的比较对象。

每个方向取32 个内部点，总共是1024 个未知量。右图数值解与解析解的最大差大约是7.54×10^{-4}。这里比较的是网格离散结果与连续问题的解析解。

现在我们已经看到了要求的场。下一步是把这个连续方程变成有限个未知数之间的关系。

第 3 页：一个五点模板，连接上千个未知量

看左上的五点模板。中心点与左右2 个邻点通过系数-1 连接，与上下2 个邻点通过负 epsilon 连接。中心的系数是$2(1+\epsilon)$。这些系数来自对二阶导数的中心差分近似。

把所有网格值按行排成向量，就得到 L x 等于 b。右端 b 包含网格步长平方与源项的乘积。式子里的 T 是1 维三对角矩阵，Kronecker 积把1 维耦合复制到2 维网格的各行、各列。

右边的矩阵图说明了这种局部连接的结果。虽然矩阵有 1000 多行、1000 多列，每行主要只和少数邻点发生关系。按当前编号排列，非零元素集中在 5 条带上；局部放大图还能分辨不同方向的系数。

这种结构直接影响计算量。每个方向64 个内部点时，未知量是 4096，非零元素只有20224 个。稠密存储需要128 MiB，而这里的 CSR 存储大约只要0.247 MiB。后面的算法会反复计算矩阵乘向量，稀疏结构让这一步的成本与非零元素数量成正比。

第 4 页：解方程等价于最小化二次能量

接下来把线性方程换成一个优化问题。我们考虑对称正定矩阵：对称表示 A 等于它的转置，正定表示任意非零方向上的二次型都严格为正。

对这样的矩阵，可以定义页面上的二次函数。它的梯度是 A x 减 b，因此梯度为 0 的点恰好满足 A x 等于 b。这就把解线性方程和寻找二次函数的最低点联系起来。

我们要区分2 个量。残差 r 等于 b 减 A x，能直接从当前解和原方程计算；误差 e 等于真解减当前解，表示我们离真解有多远。对于这个二次函数，函数值高出最低点的部分，恰好等于误差的 A 范数平方的 1/2。

右图把这种关系画成等高线。星号是最低点，椭圆的方向由特征向量决定，沿不同方向的陡峭程度由特征值决定。页面下方给出一个2 维系统，真解是 1/11 和 7/11。后面几页都用这个小例子，把抽象公式一步步算出来。

第 5 页：沿负梯度走，步长由1 维最小化确定

有了能量的解释，最直接的策略就是沿负梯度走。因为残差正好是负梯度，所以更新方向可以取 r。剩下的问题是：沿这个方向走多远?

把 x 加上 alpha 乘 r 代入目标函数，就得到只关于 alpha 的1 维函数。对 alpha 求导，第 1 项是负的 r 内积 r，第 2 项是 alpha 乘 r 内积 A r。令导数等于 0，就得到页面中间的步长公式。

看右边的曲线。对于刚才的2 维系统，初始点取 0，初始残差就是 1 和 2 组成的向量。分子是 5，分母是 20，所以最优步长是 1/4。曲线上的最低点与这个计算完全对应，第1 步的新位置就是 1/4、1/2。

这一操作还有一个重要结果：走完之后，新残差与刚才的搜索方向正交。也就是说，我们已经把当前这条直线上的能量降到了最低。接下来要问的是，为什么每一步都选到直线上的最优位置，整体收敛仍然可能很慢?

第 6 页：条件数控制最速下降的收敛尺度

关键在不同方向的曲率差异。这里用最大特征值除以最小特征值定义条件数 kappa。条件数越大，能量等高线就可能越狭长。

左图展示条件数为 20 时的前15 步。迭代在狭长谷底两侧来回转向，每一步都完成了当前方向的精确线搜索，沿谷底前进的幅度却比较有限。

右图把这种现象量化。纵轴是相对于初始值的能量误差，采用对数尺度。条件数为 2 时下降很快，条件数为 200 时曲线明显更平缓。这里选了明确的对角矩阵与初始点，使实际误差恰好按上方的收敛因子变化；左图只是把相同几何关系旋转后展示。

这个例子告诉我们，问题的难度与矩阵的谱有关。我们接下来要改进搜索方向，让后面的移动保留前面已经完成的修正，从而更有效地利用每一步计算。

第 7 页：把“方向正交”改写成能量几何中的共轭

假设我们已经沿方向 p 完成了精确线搜索，那么沿 p 的方向导数就是 0。现在再沿另一个方向 q 移动，梯度会增加 alpha 乘 A q。原来沿 p 的方向导数，也就增加了 alpha 乘 p 转置 A q。

因此，只要让 p 转置 A q 等于 0，新的移动就会保留旧方向上的最优条件。这个关系叫作 A 共轭。它使用矩阵 A 来衡量2 个方向之间的关系，曲率也随之进入了几何定义。

右边继续使用刚才的2 维矩阵。2 个坐标轴方向在普通欧氏空间中正交，但它们的 A 内积等于 1。换成 1、2 组成的 p，以及-7、6 组成的 q，就能直接算出 A 内积为 0。

右下角还给出了一个直观读法。A 的对称正定平方根把方向变换到能量坐标，变换后2 个方向的普通内积就是原来的 A 内积。于是，共轭方向在这个坐标里成为普通的直角。共轭梯度会逐步构造这样的方向，下面我们具体算2 步。

第 8 页：共轭方向保留已经完成的修正

这一页把共轭梯度完整落在2 维例子上。初始点为 0，所以残差和第1 个搜索方向都是 b，也就是 1、2 这个向量。

第1 步的步长仍然是 1/4，得到 x_1等于 1/4、1/2。接着重算残差，得到-1/2、1/4。新残差的平方范数与旧残差平方范数之比是 1/16，这就是这里的 beta。

我们把新残差与 beta 倍的旧方向相加，得到第2 个方向。请看这个式子，它保留了上一方向的信息，同时把新出现的残差加入进来。直接代入可以验证，2 个搜索方向的 A 内积恰好等于 0。

再沿第2 个方向做线搜索，步长是 4/11。把它加到第1 步的位置，就得到 1/11、7/11，正好等于真解。右边红线画出了这2 次移动，灰色虚线是最速下降的前5 步。

这个小例子让我们看到，共轭关系怎样保留之前的优化成果。接下来把这个2 维过程推广成一个逐步扩展的搜索空间。

第 9 页：Krylov 子空间把矩阵乘法变成搜索空间

从初始残差 r_0出发，先有 r_0，再有 A 乘 r_0、A 平方乘 r_0，一直往后。前 k 个向量的所有线性组合，就构成第 k 个 Krylov 子空间。

在精确算术下，CG 的第 k 个解可以理解为：在初始点加上这个子空间的范围内，选择能量最小的点。等价地，它让 A 范数下的误差最小，而且当前残差与已经搜索的子空间正交。

右边还是2 维例子。r_0是 1、2，A r_0是 6、7。这2 个向量组成的矩阵行列式为-5，所以它们线性无关，第2 步的搜索空间已经覆盖整个2 维空间。真解可以明确写成这2 个向量的线性组合，这也解释了上一页为什么2 步就能结束。

底部再换一种表示：误差可以写成一个矩阵多项式作用于初始误差。对本例，二次多项式在2个特征值处都等于 0，因此2个特征方向上的误差同时消失。这个多项式视角，会帮助我们理解下一页的谱聚集。

第 10 页：谱聚集可以大幅加速 CG

条件数只记录最大和最小特征值之间的比例。我们再看区间内部的特征值怎样分布。左图给出 2 组谱，2 组最小值都是 1、最大值都是 1000，条件数完全相同。

红色这一组按对数间隔分布在整个区间。绿色这一组除了两端的特征值，其余118个值都集中在 100 到 101 之间。对于误差多项式来说，如果在这一小段区间里取值很小，就能同时压低很多特征方向的误差。

右图是实际迭代得到的能量误差。实验让各个特征方向的初始能量相同，再用相同的真相对残差阈值停止。聚集谱用了8步，对数分布的谱用了212步。灰色虚线是只使用条件数得到的上界。

所以，除了极端特征值，我们还要关注谱的整体结构。这里使用浮点运算，迭代次数与精确算术中的有限步结论之间的关系，会在停止标准那一页继续解释。现在先把这些几何理解落实为短递推代码。

第 11 页：CG 用短递推保存方向搜索的成果

左边是算法公式，右边是相同过程的代码。初始化时计算残差，把搜索方向设为残差，并保存残差的平方范数 rr。

每轮先计算 q 等于 A p。随后用旧残差平方范数除以 p 内积 q，得到步长 alpha。这个步长同时用于更新解和残差。接着计算新残差的平方范数，与旧值相除得到 beta，再把新残差与 beta 倍旧方向组合成下一方向。

代码用 rr 的平方根检查递推残差，因此停止判断复用了已经算好的内积。每轮核心计算包含一次稀疏矩阵向量乘法和2次内积，外加几个向量更新。我们也不需要保存所有历史方向，只维护当前这些向量。

循环结束后，代码重新计算 b 减 A x。这一步直接回到原方程检查当前解。后面还会展示 SciPy 的完整停止接口。

到这里，共轭方向让搜索过程更高效。下一步，我们进一步利用矩阵的结构，改善算法所面对的能量几何。

第 12 页：预条件通过改变坐标，改善能量几何

预条件的出发点是选择一个容易求解的对称正定矩阵 P。把它写成 C 乘 C 的转置，再做页面上的变量变换，就得到一个等价系统。这个新系统仍然对称正定，同时可能拥有更好的特征值分布。

看 2 幅等高线。原系统的2个坐标尺度差别很大，左边的等高线十分狭长。这里用对角矩阵做缩放后，右边的等高线变得更均衡，条件数从大约 134 降到了 3。

实际计算时，我们可以在原坐标中实现这个变换。预条件共轭梯度，也就是 PCG，每轮先求 z 等于 P 的逆作用在残差 r 上，再使用这个经过处理的残差更新搜索方向。收敛界对应的是变换后系统的条件数。

选择 P 时有2个同时要考虑的问题：它能多大程度改善几何，以及每次应用它需要多少计算。下面就用 2 种预条件，分别处理变量尺度与网格中的强耦合。

第 13 页：预条件同时处理尺度与耦合

我们回到最开始的网格矩阵 L，再做对称缩放，构造教学系统 A 等于 S L S。S 的对角元素沿每行从 1 逐渐增加到 20，这给系统加入了明显的变量尺度差异。

左边是 Jacobi 预条件：只取 A 的对角元。应用它的时候，每个残差分量除以对应的对角元，就得到新的分量。这个操作非常便宜，而且能有效消除尺度差异。在 32 乘 32 的网格上，条件数由大约 16895 降到 441。

右边的行块预条件进一步保留横向的强耦合。它把同一行的未知量放在一起，求解一个三对角系统。由于所有网格行使用相同的结构，长度为 m 的因子可以重复使用。这个例子里，预条件后的条件数进一步降到 5.35。

下面的流程图按顺序表达了实际计算：先除以缩放向量，再逐行求解，最后再除一次缩放向量。这里实现的是 P 的逆作用。SciPy 的 M 参数接收的正是这个操作，接下来我们看对应代码。

第 14 页：行块预条件：一个带状因子服务所有网格行

这一页把行块预条件拆成可以直接实现的步骤。因为 P 由两侧的缩放矩阵和中间的行块矩阵组成，所以它的逆作用也可以分成3步：先处理尺度，解各个行块，再恢复尺度。

左侧代码中的 band 只使用 2 行。第 1 行保存主对角元，第 2 行保存次对角元，随后对这个带状矩阵做 Cholesky 分解。我们把分解放在函数外面，因此它只需要准备一次。

进入 apply_Pinv 后，先把 r 逐项除以 s，再恢复成网格形状。这里的转置使每列对应一个网格行的右端，带状求解器就能一次处理这些右端。最后再转回原顺序，并除以 s。

右边说明 PCG 对普通 CG 递推的修改。先求 z，并以 z 初始化方向；把原来的残差平方范数替换为 r 与 z 的内积 rho。alpha 和 beta 使用 rho 更新，新的搜索方向则由新的 z 与旧方向组合。

每轮要解 m 个长度为 m 的三对角系统，总工作量与 m 平方，也就是未知量个数 n 成正比。现在我们把这 2 种预条件放到同一个系统中比较。

第 15 页：同一系统上的 4 种迭代轨迹

这组比较使用完全相同的矩阵、右端和初始点。网格大小是 32 乘 32，真解由固定随机种子生成，再用 b 等于 A 乘真解构造右端，所以实验里既能计算残差，也能计算真实的能量误差。

左图是真相对残差，右图是相对能量误差。2 幅图都使用对数坐标，方法的颜色保持一致。可以看到，普通 CG 比最速下降更快；加入对角预条件后进一步加速；保留强横向耦合的行块预条件下降最快。

下方表格给出同一停止阈值下的结果。CG 用了587步，Jacobi-PCG 用了140步，Line-PCG 用了14步。最速下降运行到10000步时，真相对残差约为10^{-5}，因此表里将达到目标的步数记为大于 10000。

这组实验把前面的 2 条机制联系起来：方向共轭改善搜索，结构化预条件改善问题本身。接下来扩大网格，看看这些收益怎样变化。

第 16 页：网格加密后的收益与成本

现在把每个方向的内部点数依次取 16、32 和 64，对应 256、1024 和4096 个未知量。其他主要构造方式和停止阈值保持一致。

左图显示 3 种方法的迭代次数都随网格加密增加。CG 从225步增加到1236步，Jacobi-PCG 从72步增加到214步，行块方法则从9步增加到23步。

右图补充了预条件后条件数的变化。行块方法的条件数从 2.14 增加到 17.94，这解释了为什么它的迭代次数也在增长。

接下来要把迭代次数换成实际工作量。每种方法都需要稀疏乘法，Jacobi 还需要逐项除法，行块则需要带状求解。总成本由准备阶段加上迭代次数乘每步成本组成。同一矩阵如果对应多个右端，准备好的预条件和分解因子还可以重复使用。

第 17 页：迭代数与每步成本，共同决定总时间

这一页直接测量4096个未知量的系统。表格把准备和求解分开，结果取本机预热后7次测量的中位数。

左图横轴采用对数尺度，点的位置表示一次求解时间。CG 大约是52毫秒，Jacobi-PCG 约11毫秒，行块 PCG 约3.7毫秒。稀疏 LU 完成分解后，一次求解大约需要0.46毫秒；它前面的准备阶段则需要约8.7毫秒。

右图用准备时间加上 R 倍单次求解时间，估算同一矩阵重复求解的总成本。随着右端数量增加，准备成本被摊薄，复用分解因子的收益更明显。

因此，求解器选择需要同时看矩阵规模、每步操作和右端数量。迭代次数提供了重要线索，而实际时间把预条件应用与因子准备这些工作也一起算了进去。最后，我们还需要可靠地判断一次求解什么时候完成。

第 18 页：用真残差停止，再核对误差尺度

先看左边。SciPy 接口中的相对阈值与绝对阈值共同确定收敛标准。返回后重算真残差，按它的范数不大于相对阈值乘右端范数与绝对阈值中的较大者进行验收。相对阈值控制相对于右端尺度的精度，绝对阈值提供固定的残差尺度。

右边把残差与解误差联系起来。相对解误差的一个通用上界，是条件数乘以相对残差。对于这里条件数约 16900 的系统，相对残差达到10^{-8}，对应的这个上界约为1.69×10^{-4}。

下面代码把行块的逆作用包装成 LinearOperator，传给 cg 的 M 参数。返回后，我们同时检查 info 和重新计算的残差。本例 info 等于 0，真相对残差约为9.05×10^{-9}。

最后回到第 10 页的120维例子。精确算术中，CG 至多经过 n 步结束；浮点运算中的舍入会逐渐损失共轭性，因此那个例子用了212步。递推残差方便每一步更新，最后重算真残差则直接检查原方程。

第 19 页：直接实现五点乘法，也能接入同一个 CG 接口

到这里，我们一直把 A 看作显式存储的稀疏矩阵。CG 实际使用 A 的关键操作，是矩阵乘向量。对于最开始的五点结构，可以直接写出这个操作。

看左边的函数。先将向量按 s 缩放，并恢复为2维网格 U。V 先接收中心点的贡献，然后通过 4 组切片累加左右、上下邻点的贡献，最后再缩放并展平成向量。

这些切片只处理内部邻点。当前采用零边界条件，边界外的场值为 0，所以这一实现与原来的五点矩阵对应。把函数包装成 LinearOperator 后，下面的 cg 调用仍然使用相同的预条件接口。

右侧给出了实际核对：同一测试向量下，算子乘法与显式矩阵乘法的相对差约为1.67×10^{-16}；配合行块预条件，真相对残差达到9.05×10^{-9}。

这一步把矩阵的局部结构直接转成了计算过程。算法接口、预条件与残差检查都得以保留，存储主要用于向量和预条件数据。

第 20 页：把结构、收敛与成本连成一条求解路径

最后把整条路线合起来。面对对称正定的稀疏系统，可以从 CG 或 PCG 出发；尺度差异明显时，先考虑对角均衡；存在强方向耦合或变量组结构时，再考虑行块或其他分块方式。多个右端则需要把因子复用纳入总成本。

今天最重要的3个联系是：条件数把能量几何连接到收敛尺度，共轭方向保留已经完成的修正，预条件把矩阵结构转化为更有效的迭代。

从网格上的局部方程，到能够运行并检查结果的求解程序，我们始终围绕这 3 条线索选择表示、算法与实现。谢谢大家。

从传感器数据到可信频谱 讲稿

SKILL 应用测试

第 1 页 从传感器数据到可信频谱

今天从一段具体的传感器记录出发，讨论怎样得到可以解释的频谱。面对一条随时间起伏的曲线，我们通常想知道：有哪些振动持续存在，它们的强度是多少，以及某次短时扰动发生在什么时候。这 3 个问题分别对应频率、幅值或功率，以及时间定位。

后面的例子始终围绕同一段可复算信号展开。我们先确定采样参数，再理解离散傅里叶变换与 FFT，随后处理记录长度、窗函数和数值校准，最后用时频图与频带包络完成诊断。每一步都把图上的形状与一个明确的操作联系起来。

第 2 页 先分清信号里有哪些变化

先看左图，它是完整的 4 s 记录。波形看起来很密，因为里面叠加了多种频率。右图只展示最前面的 80 ms，可以看见合成波形与稳定分量之间的差别。稳定分量有 120 Hz、123 Hz 和 420 Hz；此外还有从 500 Hz 逐渐升到 700 Hz 的扫频，以及中心位于 2.6 s 附近的 900 Hz 短时事件。

数据还包含标准差为 0.08 V 的白噪声，基线从 0.30 V 缓慢变化到 0.35 V。这些成分解释了波形的背景起伏和偏置。采样率是 2048 Hz，连续记录 4 s，所以共有 8192 个采样点。我们接下来会反复利用这几个已知条件，检查分析结果能否找回稳定频率、恢复强度，并定位短时事件。

第 3 页 采样参数决定数据能回答什么

沿着上方流程从左向右看，模拟信号先经过抗混叠滤波，然后进行采样和量化，最后得到数字序列。这里的顺序有实际含义：进入采样器之前，就需要控制模拟信号中保留的频率范围。采样后看到的序列，是这些模拟变化在离散时刻留下的数值。

表格把 4 个常用参数放在一起。2048 Hz 表示每秒取 2048 个样本，相邻样本间隔约 0.488 ms。4 s 记录对应 8192 点；频率栅格间隔等于记录时长的倒数，所以是 0.25 Hz。采样率的 1/2 是 1024 Hz，即本例的 Nyquist 边界。采集时同时考虑目标最高频率和滤波器的过渡带，分析时再根据相近频率的间隔选择记录长度。

第 4 页 混叠让高频落到另一个位置

这页展示混叠最直接的原因。左图中，红线是 1350 Hz 余弦，青线是 698 Hz 余弦。连续曲线不同，但蓝色采样点恰好落在 2 条曲线的相同位置。因为 1350 加 698 等于 2048，在这一采样率下，这 2 条余弦在采样时刻给出同样的数值。

右图把输入频率和观测频率的关系画出来。频率越过 1024 Hz 后会向低频折回；图中的 1350 Hz 对应 698 Hz。计算时可以从输入频率中减去适当的采样率整数倍，再取绝对值，使结果落入 0 到采样率 1/2 的范围。实际采集要在模拟端先做好抗混叠滤波。这样频谱中的低频峰，才能对应我们希望保留的物理成分。

第 5 页 降采样前先限定保留的频段

现在把采样率从 2048 Hz 降到 512 Hz，也就是每 4 点保留 1 点。这页单独构造了 120 Hz 与 900 Hz 的双音信号，便于看清操作后发生的变化。左图采用直接抽取，120 Hz 峰保留下来，900 Hz 却折到 124 Hz，形成幅值 0.500 V 的假峰。2 个峰靠得很近，很容易影响对低频结构的判断。

右图使用 resample_poly，先低通再降采样。124 Hz 位置的残余幅值降到约 5.69 乘 10 的负 5 次方 V，而 120 Hz 仍清楚保留。2 幅图给出同一个操作原则：降低采样率之前，先限定要保留的频段。如果当前任务是定位 900 Hz 短时事件，就继续使用原始采样率的数据。频段选择跟随分析目标。

第 6 页 离散傅里叶变换 衡量频率基底的匹配

离散傅里叶变换，也就是 DFT，可以理解为把信号依次与一组不同频率的基底进行匹配。左图展示 32 个样本中的 5 周期信号，并放入 6 周期基底作为对照。右图是变换系数的大小，能量集中在频点编号 k 等于 5 的位置。

公式中，n 从 0 取到 N 减 1，是时间样本的序号；k 是频点编号；j 是虚数单位。将每个样本乘以相应的复指数再求和，得到复系数 X[k]，其中包含幅值和相位信息。物理频率由 k 乘采样率再除以 N 得到。对实数信号，rfft 返回非负频率部分，保留了我们后面绘制单边频谱需要的系数。

第 7 页 快速傅里叶变换 把 DFT 分解成蝶形

FFT 是快速计算 DFT 的一类算法。左图以 8 点变换为例：输入先按位反转顺序排列，再经过 2 点、4 点和 8 点的 3 级组合。每个蝶形接收 u、v 这 2 个输入，输出 u 加 ωv，以及 u 减 ωv；ω 是相应的复数旋转因子。图中的交叉连接表示这些中间结果如何进入下一层。

右表将直接复乘数和蝶形数分别列出。1024 点直接求和按 N 平方计，共 1,048,576 次复乘；基 2FFT 有 5120 个蝶形。8192 点时，对应 67,108,864 次直接复乘和 53,248 个蝶形。一般蝶形包含 1 次复乘和 2 次复加，这里的计数包含平凡乘法。随着 N 增大，FFT 的计算规模约按 N 乘 $\log_2 N$ 增长。这里 $\log_2 N$ 表示以 2 为底的对数，8 点例子对应 3 级组合。这样的分解减少了重复计算，使长记录的频谱计算变得实用。

第 8 页 延长记录 才能看清相近频率

把 120 Hz 和 123 Hz 放在一起，两者只相差 3 Hz，幅值分别为 0.9 V 和 0.35 V。3 幅图使用同样的双音成分和 Hann 窗，只改变实际记录时长。Hann 窗平滑降低记录两端的权重。0.25 s 时，2 个分量形成一个较宽的峰；增加到 1 s 后，两峰开始清楚分开；4 s 时，峰的位置和高度更容易读取。

每个峰周围较集中的区域叫主瓣。记录时间越长，主瓣越窄。频率栅格间隔等于 1 除以 T，在这 3 种情况下分别为 4 Hz、1 Hz 和 0.25 Hz。规划测量时，要把需要区分的频率间隔与可用观察时间联系起来。固定时长提高采样率会增加样本数，而延长实际记录提供了更多区分相近频率的观察。

第 9 页 补零细化频谱采样 长记录收窄主瓣

这页比较补零和延长记录的具体效果。左图始终使用同一段 0.25 s、512 点数据。红点对应 512 点 FFT；蓝色曲线对应补零到 8192 点后得到的更密频谱采样。频率栅格从 4 Hz 细化到 0.25 Hz，原来点与点之间的形状被描绘出来，方便读取峰位或进行插值。

右图使用真正记录了 4 s 的数据。它已经包含更长的时间观察，因此 120 Hz 和 123 Hz 分别形成较窄的峰。选择参数时，可以先根据区分任务确定记录时长，再根据显示和读峰需求决定是否补零。这样就能把观察时间产生的峰宽变化，与 FFT 输出点数产生的显示变化分开理解。

第 10 页 有限记录的接缝带来频谱泄漏

实际计算总要截取有限长度的数据。截取可以看作给无限长信号乘上一个时间窗。左图把 1 s 记录的末尾接回开头：120 Hz 恰好有 120 个周期，拼接比较平滑；120.5 Hz 在拼接处出现斜率突变。横轴用毫秒展示接缝附近的细节，0 对应拼接位置。

右图可以看见频谱上的差别。整周期信号集中在对应频点，而非整周期信号的能量扩散到附近和更远的频率，这就是泄漏。矩形窗对记录内的每个样本给予相同权重。我们可以使用首尾逐渐减弱的平滑窗来降低远处旁瓣，同时在后续步骤中补偿窗对幅值与功率的影响。

第 11 页 窗函数同时改变主瓣与旁瓣

左图展示 3 种窗的时间权重。矩形窗保持恒定，Hann 窗从边缘逐渐升到中间再降下，Blackman-Harris 窗的权重更集中。右图展示归一化频率响应：靠近中心的大瓣是主瓣，后面逐个起伏的部分是旁瓣。窗的形状同时改变这 2 部分。

表中给出可比较的数值。矩形窗的主瓣零点间宽度为 2 个频点间隔，最大旁瓣约负 13.3 dB；Hann 窗对应 4 个间隔和负 31.5 dB；Blackman-Harris 窗对应 8 个间隔和负 92.0 dB。主瓣宽

度关系到相近频率的分离，旁瓣高度关系到强分量对远处弱分量的干扰。根据要辨认的现象选择窗口，再沿用相同窗口完成数值校准。

第 12 页 按窗的相干增益恢复峰值幅度

窗函数降低了部分样本的权重，因此幅值需要校准。这页输入是峰值幅度 0.9 V、恰好落在整数频点上的正弦。红色柱直接按 N 归一化；蓝色柱按窗权重的总和归一化。矩形窗的平均权重为 1，所以 2 种结果都是 0.9 V。Hann 窗的平均权重为 0.5，直接按 N 计算得到 0.45 V，校准后恢复为 0.9 V。Blackman-Harris 窗直接归一化约为 0.323 V，校准后同样恢复 0.9 V。

相干增益就是窗权重总和除以 N。计算正频率内部频点的单边峰值幅度时，将加窗 DFT 系数的绝对值乘 2，再除以窗权重总和。DC 对应 0 Hz，偶数长度序列的 Nyquist 端点对应采样率的 1/2，这 2 个端点保留单倍权重。这个规则后面会直接落实到代码中。

第 13 页 功率谱密度积分回到信号均方值

幅值谱回答某个正弦分量有多强，功率谱密度简称 PSD，它把均方量按频率进行分配，单位是 V^2/Hz。这页只使用稳定分量与白噪声。左图能看见突出于噪声背景的频率峰，右图把谱从低频开始累计积分，因此经过主要峰时会出现明显台阶。

归一化时，将加窗 DFT 的模平方除以采样率和窗权重平方和；单边谱内部频点再乘 2。所有频点的 PSD 乘频率间隔后求和，得到 0.4852 V^2。我们在时域把加窗信号的平方求和，再除以窗权重平方和，得到同一结果。这就是当前窗口和归一化下的 Parseval 一致性核对；开平方后，均方根约为 0.6966 V。

第 14 页 Welch 通过分段平均降低估计波动

噪声频谱的单次估计会有明显起伏。Welch 方法把 8192 点记录分成长度 512、重叠 256 点的片段，共 31 段。每段乘 Hann 窗、计算功率谱，再把它们平均。左图的灰线是单次周期图，蓝线是 Welch 估计，青绿色虚线表示白噪声的理论功率谱密度。平均后，背景起伏明显减小。

右图进一步展示 300 次独立白噪声实验在 800 Hz 处的估计分布。横轴是估计值除以理论值，1 附近表示接近理论水平。Welch 的分布更集中，标准差与均值之比从 0.949 降到 0.180。分段平均改善了随机估计的稳定性，分段长度同时决定每段可以观察的频率尺度。分析时一起选择片长、窗口和重叠量。

第 15 页 短时傅里叶变换定位频率随时间的变化

整个记录的频谱告诉我们有哪些频率；短时傅里叶变换，也就是 STFT，把这个操作放进滑动窗口，就能观察频率随时间怎样变化。这里窗长 512 点，即 250 ms，步长 64 点。横

轴表示窗口中心时刻，纵轴表示频率，颜色表示经过相干增益校准的局部幅度，按相对于 1 V 的 dB 显示。

现在把图和最初的信号组成对应起来。120 Hz 与 123 Hz 在这个短窗下合成低频亮带；420 Hz 是一条较独立的水平线。500 到 700 Hz 的扫频形成斜线，900 Hz 短时事件在 2.6 s 附近形成亮斑。我们可以沿着这些结构读出持续分量、频率变化和局部事件，从一张全局频谱推进到时间与频率的联合描述。

第 16 页 窗长决定时域与频域的观察尺度

这页固定时间和频率显示范围，比较窗长分别为 125 ms 与 500 ms 的 2 种设置。2 幅图的步长都为 64 点，也就是 31.25 ms，颜色范围也相同。左图的 900 Hz 短时事件在时间上更集中；右图使用更长窗口，将事件附近更大一段时间合在一次计算中，因此亮区沿时间方向展开。

相应的频率栅格分别为 8 Hz 和 2 Hz。长窗提供更细的频率观察，短窗帮助定位快速变化。步长控制相邻帧有多密，窗长决定每帧覆盖多长时间。实际选择可以从目标现象出发：关心短促冲击的起止，就重点看时间局部性；关心靠得很近的频率，则需要足够长的观察。

第 17 页 将稳定峰与短时事件分开定位

回到最初的 3 个问题，现在分别提取稳定结构与短时事件。左图使用完整 4 s 记录和 Hann 窗，120 Hz 和 123 Hz 处的 2 个峰清楚分开。长记录同时支撑近邻峰的区分与峰位读取，这和前面记录长度的比较一致。

右图专门处理 900 Hz 附近的事件。先用 850 到 950 Hz 带通滤波保留目标频带，再用 Hilbert 变换得到解析信号，取绝对值形成振幅包络。这样可以观察快速振荡外面缓慢变化的强度轮廓。包络峰位于 2.602 s，与构造事件中心 2.600 s 接近。图中的竖虚线标出 2.6 s，方便直接比较事件中心与提取结果。

第 18 页 把归一化与窗口参数写进代码

左边代码先生成 Hann 窗，再对加窗序列计算 rfft；rfftfreq 根据 N 和采样率生成对应的物理频率。系数绝对值先除以窗权重总和，然后对内部频点乘 2。对奇数长度，最后一个频点也属于普通正频率，所以再单独倍增；N 大于 1 的条件保护了单样本的 DC 情况。

右边调用 Welch，明确写出采样率、窗口、片长 512 和重叠 256。constant 表示对每段去除均值，density 表示输出功率谱密度。最后将 PSD 求和并乘相邻频率差，得到积分功率。使用代码时，先准备实数序列 x、样本数 N 和以 Hz 为单位的 fs，再按分析目标处理均值或趋势。库名 np 对应 NumPy，signal 对应 scipy.signal。

第 19 页 用可复算的检查锁定分析结果

得到频谱之后，可以用已知性质逐项复算。先做逆变换：把 rfft 结果送入 irfft，与原始序列比较，最大差约为 1.11 乘 10 的负 15 次方 V。再用已知幅值 0.9 V 的整频点正弦检查 3 种窗的幅值校准，最大差约为 1.33 乘 10 的负 15 次方 V。

功率检查将频域积分与窗加权时域均方相比较，差约为 1.11 乘 10 的负 16 次方 V^2。时间检查则把提取的 2.602 s 与构造事件的 2.600 s 进行对照。这几项分别验证变换可逆性、幅值尺度、功率尺度和事件定位。把同一采样率、窗口、归一化与单位贯穿图表和代码，就能把每个数值追溯到同一个信号及明确的计算步骤。

第 20 页 从问题出发 选定频谱分析的参数

最后把方法按问题整理。想确定最高频率范围，先选采样率并安排抗混叠滤波；想把相近的峰分开，先争取足够长的实际记录，再选择窗口；想解释幅值和功率，使用对应的归一化，并通过单位和积分检查尺度；想确定变化时刻，则结合 STFT 的窗长、步长和目标频带包络。

这条分析链从采集一直延伸到结论。采样保留目标频段的信息，窗口定义观察范围，归一化赋予图上数值明确含义，复算检查把结果与输入联系起来。面对新的传感器数据时，可以沿着这些问题逐项设定参数，再用波形、频谱和时频图互相核对。页面底部的 NumPy 和 SciPy 资料可以继续查阅具体接口与实现。

How Kalman Filters Track Noisy Motion

Speaker manuscript

SKILL Application Test

Slide 1 How Kalman Filters Track Noisy Motion

Today we will follow a moving object using a sensor that reports noisy positions. The useful output is a trajectory, a velocity estimate, and an uncertainty estimate that changes as information arrives. We will build the estimator from a small numerical example, connect that example to the matrix equations, and then test the result on a reproducible tracking problem.

The central operation repeats at every timestamp. A motion model predicts the next state. A measurement supplies a discrepancy from that prediction. The filter uses the uncertainties of both sources to determine the correction. We will also examine missing measurements, a large injected error, and repeated trials that check whether the reported uncertainty has the expected meaning.

Slide 2 Track the motion behind noisy measurements

The blue curve is the true position in a constructed motion experiment. The points are the measurements available to the estimator. We have 100 samples separated by 1 second. The position sensor adds independent Gaussian noise with a standard deviation of 3 meters. The moving object also experiences random acceleration, so the curve changes direction and speed over time.

We keep the true trajectory because it lets us measure estimation error directly. The estimator itself receives the position measurements and the specified motion model. Its tasks are to estimate position, infer velocity, and report a covariance that describes the remaining uncertainty. Read the figure as a distinction between the physical quantity we want and the measurements we actually receive. Later figures will use exactly this trajectory to compare parameter choices.

Slide 3 Represent the state and the measurement separately

The state vector contains position p and velocity v. Position is measured in meters and velocity in meters per second. A hat over a variable denotes an estimate. The sample index k tells us which timestamp we are discussing. Keeping the physical units explicit will help us interpret both covariance entries and gain components.

The sensor observes position, so its measurement matrix H is the row vector containing 1 and 0. Multiplying H by the state selects the position component. Sensor noise is then added to produce z. Its variance R is 9 square meters, the square of the 3-meter standard deviation. Velocity becomes observable through changes in position across

time. The bottom diagram shows the state-to-measurement mapping; the motion model will connect successive states.

Slide 4 A motion model carries information into the next step

The deterministic prediction uses a constant-velocity transition. Over an interval delta t, position advances by velocity multiplied by elapsed time, while the predicted velocity stays the same. The transition matrix F encodes those relationships. Random acceleration enters through G, with a position contribution of delta t squared divided by 2 and a velocity contribution of delta t.

We describe that acceleration with variance q. In this discrete model, its covariance contribution to the state is q times G G transpose. At a 1-second interval and q equal to 0.2, the process covariance Q has entries 0.05, 0.10 and 0.20 as displayed. The cross terms arise because the same acceleration affects position and velocity. Each covariance entry carries units determined by its corresponding state components. Both the dynamics and uncertainty propagation therefore need the actual elapsed time.

Slide 5 Covariance describes the shape of uncertainty

The covariance matrix summarizes the estimation error in both state components. Its diagonal entries are position and velocity variances. The off-diagonal entry measures how errors in those components vary together. A positive cross-covariance means that a larger position error tends to accompany a larger velocity error.

The contours turn that matrix into geometry. These are joint 95 percent Gaussian contours, drawn on position-error and velocity-error axes. The initial covariance is diagonal, so its principal directions align with the axes. Prediction makes the ellipse tilt because uncertain velocity affects the next position. A position measurement then changes its size and shape. This example starts from diagonal entries 4 and 1, predicts for 1 second with acceleration variance 0.04, and updates using measurement variance 9. The contour equation uses the complete state estimation error x minus x-hat.

Slide 6 First combine two scalar estimates

Before using matrices, consider a scalar position update. The prior mean is 10 meters with variance 4 square meters. The new measurement is 13 meters with variance 9 square meters. The minus superscript indicates the value before incorporating the measurement, and the plus superscript indicates the value after the update.

The gain is the prior variance divided by the sum of the prior and measurement variances. Here it is 4 divided by 13, approximately 0.308. The discrepancy is 3 meters, so the estimate moves about 0.923 meters toward the observation and becomes 10.923 meters. Its variance becomes 2.769 square meters. On the graph, the posterior combines the prior density with the measurement likelihood. Its location reflects the relative

uncertainty of the sources, and its narrower shape reflects the information gained by combining them.

Slide 7 Predict both the state and its covariance

The prediction step advances the mean with F and advances the covariance with F P F transpose plus Q. The same model acts on both the estimated trajectory and the uncertainty around it. The added Q accounts for changes that the deterministic prediction does not specify exactly.

For the worked example, start at position 10 meters and velocity 2 meters per second, with covariance diagonal entries 4 and 1. Use a 1-second interval and acceleration variance 0.04. The predicted state is position 12 and velocity 2. The predicted covariance has position variance 5.01, velocity variance 1.04 and cross-covariance 1.02. The component equation on the left explains the position-variance increase: previous position uncertainty, cross-covariance, velocity uncertainty propagated over time, and process noise all contribute. We carry the resulting state and covariance into the measurement update.

Slide 8 The innovation measures a prediction error

The innovation y is the observed measurement minus the predicted measurement. For a position sensor, the predicted measurement is the predicted position. Its covariance S includes the projected state uncertainty and the sensor variance. This gives the discrepancy a scale that is specific to the current prediction and observation.

Continue the numerical example with a measurement of 13 meters. The predicted position was 12, so the innovation is 1 meter. Its variance is 5.01 plus 9, giving 14.01 square meters. Dividing the innovation by the square root of S gives 0.267. Under the stated Gaussian model, a 95 percent reference interval for the innovation extends to about plus or minus 7.34 meters. The observed 1-meter discrepancy is therefore near the center of the predicted distribution. This normalization will later support both diagnostics and outlier screening.

Slide 9 One position observation can correct velocity too

The matrix gain projects uncertainty between state space and measurement space. Here H selects position, so P minus times H transpose is the first column of the predicted covariance. Dividing that column by S gives a position gain of about 0.3576 and a velocity gain of about 0.0728 per second.

Multiplying those components by the 1-meter innovation updates the state to position 12.3576 meters and velocity 2.0728 meters per second. The velocity update comes from the positive cross-covariance created during prediction. A position observation that is higher than expected suggests that the object may also be moving faster. The table changes the sensor variance while holding the prediction fixed. Reducing R from 9 to

1 increases both gain components. A more precise sensor has more influence on the corrected state.

Slide 10 Update uncertainty with the corrected state

Once we change the state estimate, we also update the covariance. The Joseph form uses A equal to I minus K H and combines A P minus A transpose with K R K transpose. These terms make the remaining predicted-state uncertainty and the incorporated measurement-noise contribution explicit.

The numerical covariance on the left is the prediction before the measurement. After the update, position variance is about 3.2184 and velocity variance about 0.9657. Position standard deviation falls from 2.238 meters to 1.794 meters. The off-diagonal covariance changes as well, because the observation affects the joint uncertainty. In code, the Joseph expression is useful for retaining the covariance structure through floating-point calculations. We will also remove tiny numerical asymmetry by averaging the computed covariance with its transpose.

Slide 11 The estimator is a repeating predict-update loop

Follow the diagram clockwise from the previous estimate. We begin with the corrected state and covariance from the preceding timestamp. Prediction advances both quantities. The new measurement produces an innovation and its variance. Those determine the gain, which corrects the state. The covariance update completes the cycle and supplies the next prediction.

This arrangement also makes operational behavior clear. When a measurement is missing, prediction can still advance to the next timestamp. The predicted covariance grows according to the motion model and process uncertainty. When observations return, the usual correction resumes. The carried state has a fixed size: the state vector and its covariance summarize the measurements already incorporated. Logging the full history is useful for analysis, while the recursive estimator itself only needs its current state and covariance.

Slide 12 The tracking run reduces position error

Return to the 100-sample tracking run. The left panel plots position error, defined as estimate minus truth. The shaded band shows plus or minus 1.96 times the estimated position standard deviation. The right panel shows true velocity, the inferred velocity, and the much noisier differences of successive position measurements.

For the numerical summary, we evaluate steps 10 through 99 after the initial transient. RMSE means root mean square error: square each error, take the average, then take the square root. The measurement position RMSE is 3.133 meters and the filtered position RMSE is 2.143 meters. The interval is a different output from the trajectory error. It reports the uncertainty predicted by the model; later repeated trials will test

its calibration. Keeping both views helps us assess the estimate and the uncertainty it carries.

Slide 13 Velocity is inferred through motion over time

A direct velocity estimate divides the difference between consecutive position measurements by the elapsed time. Each difference contains noise from both observations. For independent sensor noise, the variance of this contribution is 2R divided by delta t squared. With R equal to 9 and delta t equal to 1 second, that is 18 square meters per square second.

In the tracking run, the direct-difference velocity RMSE is 4.204 meters per second. The velocity component of the Kalman state has RMSE 0.848 meters per second. The model combines position changes over time with uncertainty information. The small observability matrix at the bottom explains why this can work. H sees position immediately, while H F includes a velocity term multiplied by elapsed time. With nonzero elapsed time, these rows have rank 2, allowing both state components to affect observations across time.

Slide 14 Process noise controls responsiveness

The process covariance expresses how much motion can change between observations. Here we vary the assumed acceleration variance q while keeping the measurements fixed. The generating process uses q equal to 0.2. The gray estimate uses 0.002, the red estimate uses 0.2 and the green estimate uses 2.

The left plot shows position error and the right plot shows a segment of the velocity trajectory. A smaller q makes the estimate persist more strongly with the model, producing a smoother response that can lag changing motion. A larger q permits more rapid movement in the estimate. The table quantifies this run: position RMSE is 4.401, 2.143 and 2.445 meters for the 3 settings. Velocity RMSE is also lowest at q equal to 0.2. Use such controlled comparisons to connect the tuning parameter to the behavior that matters in the application.

Slide 15 Read the innovation and gain as diagnostics

The standardized innovation compares each discrepancy with the uncertainty predicted for that comparison. The dashed references are at plus and minus 1.96. This makes the residual sequence easier to interpret over time than raw errors alone. Persistent shifts or sustained changes in its scale can point to a changed sensor or a changed motion regime.

The right panel shows the position and velocity gain components. Their initial movement reflects the initial covariance and the information arriving from the first observations. They then settle as the covariance recursion approaches a stable regime for the repeated model and measurement configuration. These curves make the estimator

inspectable: the innovation describes incoming disagreement, while the gain shows how strongly that disagreement changes the state. The velocity gain has units of inverse seconds, matching the conversion from position discrepancy to velocity correction.

Slide 16 Missing measurements increase uncertainty

The gray interval marks a gap in observations at steps 40 through 50. During this gap, the filter keeps applying prediction. Its trajectory continues according to the current motion estimate, and the covariance accumulates the effect of uncertainty in velocity and acceleration.

The position standard deviation is 1.944 meters at step 39. By step 50 it has grown to 13.863 meters. When the next observation arrives, the correction reduces it to 2.944 meters. Read the left and right panels together: the first shows the predicted path during the gap, and the second shows how uncertain that path has become. This is why missing observations need an explicit branch in an implementation. The estimator still advances in time, and its uncertainty reflects the information that has not arrived.

Slide 17 Use the innovation scale to screen a large outlier

For this test, add 30 meters to the observation at step 65. The gray curve accepts every update and is pulled away from the true trajectory. The red curve uses an innovation gate. Its decision is based on normalized innovation squared, or NIS, which is y squared divided by S for this scalar measurement.

Under the stated Gaussian measurement model, NIS has a chi-squared reference distribution with 1 degree of freedom. The 99 percent threshold is 6.635. The injected observation has NIS about 38.42, so the gate skips that correction and keeps the prediction at this timestamp. The following observations resume normal updating. This establishes a concrete handling policy for unusually large discrepancies. The threshold, model assumptions and behavior after a rejected update should all be explicit in the implementation.

Slide 18 Check covariance calibration across repeated trials

Repeated trials show how often the true position lies inside the reported uncertainty intervals. Here we run 400 independent trials, evaluate 90 steps from each, and compare different assumed sensor variances. The generating model uses q equal to 0.2 and R equal to 9.

The left panel reports mean NIS. With the matched sensor variance, it is 1.007, close to the scalar reference value of 1. The right panel reports interval coverage: the fraction of true positions falling inside the estimated 95 percent position intervals. The matched case reaches 95.1 percent. Assuming R equal to 1 produces narrow, overconfident intervals and coverage of 51.7 percent; assuming 36 produces broader intervals and

coverage of 99.8 percent. These are measurable consequences of the covariance choices. Calibration checks connect reported uncertainty to repeated outcomes.

Slide 19 Translate the equations into a reusable update

The function starts with prediction for x and P. If z is missing, it returns those predicted quantities. Otherwise it computes the innovation y and its covariance S. The gain is obtained by solving a linear system. This expresses the same gain equation while avoiding the need to form a separate matrix inverse explicitly.

The state update uses K times y, and the covariance update uses Joseph form. The final symmetrization removes small numerical discrepancies between matching off-diagonal entries. Array shapes matter: the state has length 2, H is 1 by 2, the measurement has length 1, and R is a 1 by 1 matrix. The caller builds F and Q for the elapsed time and applies any observation-gating policy. Keeping that contract explicit makes the update usable in a stream of observations with a clear state representation.

Slide 20 Build the estimator around a testable motion model

The design decisions now form a complete chain. Choose physically meaningful state variables and units. Define how the sensor observes them. Build the transition and process covariance from the time interval and motion assumptions. Set the initial uncertainty, characterize measurement noise, and use observed errors and innovations to assess tuning.

The operating rules also belong in the design. Specify how prediction advances through gaps and how unusually large innovations are handled. Validate the estimated trajectory against known truth when available, and test uncertainty coverage across repeated trials. The main idea to retain is that the filter carries an estimate and a quantitative description of its uncertainty together. Prediction evolves that pair through the model, and measurement updates revise that pair using new information.

How Indexes Accelerate Queries

Speaker manuscript

SKILL Application Test

Slide 1 How Indexes Accelerate Queries

We will use a concrete query to understand what an index changes inside a database. The query asks for the 20 most recent orders belonging to a particular tenant. Its result stays fixed while we change the available index. This lets us connect a data structure to a query plan and then to the work performed during execution.

The story begins with a full scan. We then add a tenant key, combine that key with the requested order, and extend it to supply the output values. Along the way, we will look at column order, partial indexes, expression predicates and maintenance costs. The goal is to choose an index by understanding the work that the query actually needs.

Slide 2 Start with one query and a fixed result

Read the SQL from the filter outward. We select orders for tenant 42, order them by descending creation sequence, and return the first 20 values of created_seq and amount_cents. The creation sequence is an increasing integer that represents creation order. The amount is stored as integer cents. Each row also has a 64-byte payload field.

The experiment contains 200,000 synthetic orders spread across 100 tenants. Tenant 42 has 1,980 rows. We measure SQLite virtual-machine work using a progress callback at an instruction interval of 1, and separately time complete result retrieval after warm-up. Every comparison checks the returned rows and their order. Keeping that check in place makes a change in execution work interpretable as an access-path change for the same requested answer.

Slide 3 An index stores an additional order over the data

These small illustrative tables show the idea before we examine a larger tree. The table on the left is arranged by row ID. The index on the right is arranged first by tenant and then by row ID. The same logical record can therefore be located through a different ordering.

For tenant 42, the corresponding index entries are adjacent. After finding those entries, their row IDs identify the table records that contain additional values. This gives the index 2 roles: its leading key helps narrow the search, and its row identifiers connect the search result to the remaining table data. The exact order of key fields becomes important when a query includes several predicates or requests a particular output order.

Slide 4 A B tree routes the search through a few pages

The diagram represents storage pages as nodes. The upper page contains separator keys 30 and 60. A search for 42 follows the middle range because 42 lies between those separators. The selected lower page then contains the matching entry.

A real database page can hold many keys and child links. That branching capacity is called fan-out. It lets a tree represent a large number of entries while keeping the route from the root to a

target region short. The diagram uses a small number of illustrative keys so that the route is easy to trace. The next page will connect this hierarchical idea to the actual page inventory measured for an index in our SQLite experiment.

Slide 5 High fan out keeps the tree shallow

The measured index has 6 internal pages and 846 leaf pages. Its longest root-to-leaf path contains 2 edges, so the tree has 3 levels. The internal pages contain 845 entries and the leaves contain 199,155 entries, totaling the 200,000 indexed records. This page inventory comes from SQLite's dbstat view.

The storage figures distinguish the small internal routing structure from the much larger leaf region. The internal pages occupy 24 KiB, while the leaves occupy about 3.305 MiB. A selective lookup follows the tree toward a relevant key region. A full scan instead traverses the chosen table or index across its entries. That distinction prepares us to read SCAN and SEARCH in the execution plans that follow.

Slide 6 Without an index filter and sort the table

The baseline has no secondary index for the tenant predicate. Follow the sequence in the diagram: inspect the table rows, identify the 1,980 rows belonging to tenant 42, and determine which 20 have the largest creation sequences. The result is small, but finding the eligible recent rows requires substantial work.

EXPLAIN QUERY PLAN reports a scan of orders and a temporary B-tree for ORDER BY. These descriptions expose the broad access path and the separate ordering work. In the measured run, the query executes 617,851 virtual-machine instructions to retrieve its 20 rows. We will keep this baseline result and compare it with each index design. The diagram and plan provide the mechanism behind the number.

Slide 7 A tenant index removes most filtering work

The first added index contains tenant_id. It makes all entries for the selected tenant accessible through an equality search. The plan now reports SEARCH orders USING INDEX idx with a tenant equality constraint. The question mark in the plan describes the equality lookup value.

The query still requests descending creation sequence, while this index is ordered by tenant and row ID. It therefore needs table records for the remaining values and separate ordering work. The temporary B-tree for ORDER BY remains in the plan. Even so, the measured instruction count falls to 23,792. We have removed much of the filtering work by narrowing the search to a tenant, while the requested order gives us the next design opportunity.

Slide 8 A composite index groups and orders together

A composite key uses more than one field. The index shown here starts with tenant_id and then uses created_seq. Read the illustrative rows lexicographically: tenant is compared first; sequence determines order among entries whose tenant values are equal.

This places tenant 42 in one contiguous region. Within that region, creation sequences are ordered from smaller to larger values. The equality constraint fixes the leading key, so the following sequence field can supply the order requested by the query. This is the important relationship

between predicates and ordering: the meaning of a later key field depends on which earlier key fields are fixed by the query.

Slide 9 An ordered index can stop after the first 20 matches

Once the tenant range is ordered by creation sequence, the database can seek to its most recent end and walk entries in descending order. It returns rows as it goes and stops after 20 matches. The requested sort order now follows directly from the index traversal.

The plan reports the tenant search without a separate temporary B-tree for ordering. Measured work falls to 150 instructions. Compare this with the earlier plan that gathered tenant rows and then determined their order. The combination of the equality predicate, the key order and LIMIT allows early termination. This explains the large reduction in work even though both plans use an index and return exactly the same values.

Slide 10 A covering index supplies every requested value

The next index adds amount_cents after tenant_id and created_seq. Each field now has a role in the query: tenant supplies the equality search, sequence supplies order and an output value, and amount supplies the other output value.

The plan reports USING COVERING INDEX because the required values can be obtained directly from the index. Table lookups for the selected amounts are removed. The measured count drops from 150 to 129 instructions. This is a refinement of an already selective and ordered path. The much larger earlier reductions came from avoiding a full scan and a separate sort; coverage removes the remaining table-access work for this query.

Slide 11 Column order determines the searchable prefix

This page switches to a range aggregate. The SQL sums amount_cents for tenant 42 over creation sequences 80,000 through 119,999. Both compared indexes contain tenant and sequence, but their order is reversed. INDEXED BY selects the intended path so that the experiment isolates that ordering decision.

With tenant first, the search enters one tenant's key range and narrows it by the sequence bounds. With sequence first, it enters the sequence range across tenants and then applies the tenant condition. The results agree, while the instruction counts are 1,991 and 200,808. The diagram-free table makes the comparison precise: a list of indexed columns is only part of the design; their order determines the constrained region that a traversal can use.

Slide 12 Coverage depends on the requested output columns

Keep the index containing tenant, sequence and amount, and compare 2 output lists. The first asks for sequence and amount, both present in the index. The second also asks for payload, which is stored in the table record.

That extra output changes the observed plan from USING COVERING INDEX to USING INDEX. The query still benefits from the tenant search and sequence order, but now it follows row IDs to fetch payload. The measured instruction count is 170 for this version. Coverage should therefore be evaluated against the actual query, including the values it needs. A change to the

requested output can change the access work even when the WHERE and ORDER BY clauses remain the same.

Slide 13 A partial index represents a useful subset

A partial index contains rows that satisfy a specified predicate. Here the index contains tenant, sequence and amount for rows whose status is open. The workload has 20,045 open orders, roughly one tenth of the complete table.

The query in this comparison explicitly includes status equal to open, so the partial index contains every row that can qualify. We compare it with a full status-first index containing status, tenant, sequence and amount. The partial index occupies about 0.336 MiB, while the full index occupies about 4.641 MiB. Both return the same active-order result. This makes the index predicate a useful part of the physical design when the application repeatedly queries a defined subset.

Slide 14 An expression can change the usable search constraint

The 2 predicates on this page return the same count in this integer dataset. The direct equality tenant_id equals 42 uses a constrained search. Wrapping the column in tenant_id plus 0 leads to a scan of the covering index in the observed plan.

The measured counts are 5,953 instructions for the direct equality and 801,994 for the expression predicate. Both return a count of 1,980 matching rows. This comparison shows why query form matters alongside index structure. When designing a query, keep the intended constraint directly searchable, or build an index for the expression that the application genuinely needs. The useful check is the resulting plan and measured work for the actual expression.

Slide 15 Read the query plan as a physical route

We can now interpret the main plan terms through examples we have already seen. SCAN describes traversal through a selected table or index. SEARCH identifies a constrained lookup, such as the tenant equality. USING COVERING INDEX tells us that the index supplies the values required by the query. USE TEMP B-TREE FOR ORDER BY identifies separate ordering work.

Read these details together rather than treating an index name as the complete result. A scan can itself traverse an index, as the expression example showed. A search can still require table lookups and sorting, as the tenant-only example showed. Pair the plan with the returned rows and the work measurements. The result confirms semantics, the plan explains the route, and the measurements quantify what that route did.

Slide 16 The same result can require radically different work

This chart collects the 4 versions of the original ordered, limited query. The horizontal axis is logarithmic, so each major step represents a factor of 10. The bars are also labeled with their measured instruction counts, which makes the smaller values directly readable.

The full scan executes 617,851 instructions. A tenant index reduces that to 23,792. Matching tenant and sequence order reduces it to 150. Adding the selected amount column gives a covering path with 129 instructions. All 4 versions return identical rows in identical order. The sequence of changes explains the outcome: first narrow the region, then obtain the order directly, then supply the remaining output values inside the index.

Slide 17 Latency and storage reveal different costs

The timing experiment uses SQLite 3.53.1 with an in-memory database containing 200,000 rows. Each plotted time is the median of 11 executions after 3 warm-up runs. The left axis is logarithmic, which allows the scan and the much shorter indexed paths to share one chart.

The right panel shows additional allocated database storage for the index designs. Another key order requires additional pages. Adding output fields to make an index covering also increases its entry width. Read the 2 panels as different dimensions of the design: one describes the observed time to retrieve the query result in this environment, and the other describes the storage needed for the added structures. The deterministic instruction counts from the previous page help connect timing to execution work.

Slide 18 Index maintenance and broad reads also consume work

An index is maintained when data changes. The left panel inserts the same 5,000 rows in one transaction for each index configuration and counts virtual-machine instructions. The transaction is rolled back after measurement so that the comparisons start from the same underlying data.

The right panel changes the fraction of rows selected by a sequence-range query and fetches payload from the table. Each access path is selected explicitly for the comparison. A small range lets the index narrow the work effectively. As the requested fraction grows, more entries and records must be visited. At 100 percent, both paths traverse the complete dataset. These measurements connect index design to the read and write operations that make up the workload.

Slide 19 Build indexes through a repeatable query experiment

Begin by writing down the exact query contract: its predicates, ordering, selected values and row limit. Capture the baseline result and plan. Then change the key design to match the operations that the query requires. This keeps the index proposal tied to a specific access path.

Measure the result on several dimensions. Check result equivalence, read work, storage and write maintenance. Refresh planner statistics with ANALYZE or the appropriate PRAGMA optimize workflow and inspect the resulting plan again. Statistics help the planner estimate the choices available to it. Retain these measurements when the query, schema or data distribution changes, so that later adjustments can be compared with a clear baseline.

Slide 20 Choose the index that matches the work to be done

The final matrix connects query requirements with index properties. Selective equality benefits from a usable leading key. Ordered, limited results benefit when later key fields provide the requested order after earlier fields are fixed by equality. Covering access supplies required values directly. A partial index represents a subset described by a predicate that the query satisfies.

Read and write patterns determine how these properties combine into a useful design. The experiments showed that a small returned result can require very different work depending on search, ordering and table lookups. Keep that physical route in view. An index helps by narrowing, ordering or supplying data along the path that the query actually follows.